

Programiranje za web

Predavanje 4: Autorizacija i vlasništvo nad zapisom

doc. dr. sc. Tomislav Petković mag. ing. Josip Torić

29.10.2026.



Sveučilište u Zadru
Universitas Studiorum
Jadertina | 1396 | 2002 |

Plan predavanja

- 1 Što autorizacija odlučuje
- 2 Provjera uloge
- 3 Vlasništvo nad zapisom
- 4 Odgovori i njihove posljedice

Na kraju predavanja svaka ruta zna tko smije poslati zahtjev i nad čijim podacima.

Ishodi učenja

Nakon ovog predavanja student može:

- razgraničiti provjeru uloge od provjere vlasništva i reći koju vrstu zaobilaženja svaka sprječava
- napisati ovisnost koja propušta samo određene uloge
- provesti provjeru vlasništva u sloju poslovnih pravila i obrazložiti zašto ne stoji u ruti
- razlikovati odbijanje zahtjeva od sužavanja odgovora
- obrazložiti izbor između odgovora 403 i 404

Domena primjera

Sustav prijava klubova na powerlifting natjecanje. Račun ima ulogu `admin` ili `club`. Svaki klub ima jedan račun koji pripada samo njemu; organizator ne pripada nijednom.

Iz prethodnog predavanja svaki zahtjev nosi token, a ovisnost `get_current_user` iz njega dobiva zapis o korisniku iz baze. Danas se odlučuje što taj korisnik smije.

Što autorizacija odlučuje

Granica između identiteta i prava, uloga i vlasništvo, modeli autorizacije.

Od identiteta do prava

Znati tko šalje zahtjev nije isto što i znati smije li ga poslati. Primjer iz domene:

- Klub pokušava stvoriti drugi klub. Zahtjev je valjan i pošiljatelj je poznat, ali stvaranje klubova nije njegov posao nego organizatorov.
- Klub A dohvaća podatke kluba B. Za tu radnju ima pravo, ali samo nad zapisima svojega kluba.

Prvi slučaj ovisi samo o tome tko je pošiljatelj. Drugi ovisi i o tome na koji se zapis zahtjev odnosi, pa se ne može odlučiti dok se ne zna o kojem je zapisu riječ.

Zato autorizacija ima dva dijela, i oni se provode na različitim mjestima.

Uloga i vlasništvo

Provjera	Odgovara na	Sprječava
uloga	smije li ovakvu radnju	radnju iznad svojih prava
vlasništvo	smije li nad ovim zapisom	dohvat tuđih podataka

Uloga je svojstvo korisnika i ne ovisi o zahtjevu: klub nikada ne stvara klubove.

Vlasništvo je odnos između korisnika i zapisa: isti korisnik smije mijenjati jedan zapis, a drugi ne, premda je radnja ista.

Iz te razlike slijedi i mjesto provjere: uloga se može provjeriti prije nego bude dohvaćen zapis na koji se zahtjev odnosi, vlasništvo tek nakon toga.

Modeli autorizacije

Model	Odluka se donosi prema	Primjer
ACL	popisu uz svaki zapis	prava nad datotekom
RBAC	ulozi korisnika	organizator i klub
ABAC	svojstvima korisnika, zapisa i okolnosti	pravo koje ovisi o dobu dana

Redom: popis prava uz svaki zapis (engl. *access control list*), prava vezana uz ulogu (engl. *role-based access control*), prava izračunata iz svojstava (engl. *attribute-based access control*).

Ovdje se koristi RBAC uz provjeru vlasništva, jer su uloge dvije i pravila se ne mijenjaju bez izmjene koda. ABAC dolazi u obzir kad pravila ovise o okolnostima ili ih treba mijenjati bez nove inačice aplikacije; ACL kad se prava dodjeljuju pojedinačno, po zapisu.

Provjera uloge

Odakle dolazi uloga, kako se provjerava, uporaba u ruti.

Odakle dolazi uloga

Uloga mora biti poznata pri svakom zahtjevu, a doći može s dvaju mjesta:

Izvor	Kada promjena vrijedi	Cijena
iz tokena	tek kad token istekne	nema upita
iz baze	odmah	upit po zahtjevu

Token nosi staro stanje

Token nosi ono što je vrijedilo u trenutku izdavanja. Oduzme li se korisniku pravo, token izdan prije toga i dalje nosi staru ulogu dok ne istekne. Zapis u bazi nosi novu vrijednost odmah, ali za njega treba upit.

Izbor, ne pravilo

U našem se projektu uloga čita iz baze, jer `get_current_user` zapis o korisniku ionako dohvaća da provjeri je li račun aktivan; upit je time već plaćen. Aplikacije s vrlo velikim brojem zahtjeva odlučuju iz tokena i prihvaćaju da promjena prava kasni do njegova isteka.

Kako provjeravamo ulogu

```
# app/core/deps.py
def require_role(*allowed_roles: str):
    def checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in allowed_roles:
            raise AppError("forbidden", "Nemate dozvolu za ovu akciju", 403)
        return current_user
    return checker
```

`require_role` nije ovisnost nego funkcija koja je stvara. Poziv `require_role("admin")` vraća funkciju `checker`, a nju ruta navodi u `Depends`. Tako jedan zapis pokriva sve kombinacije dopuštenih uloga. Unutar sebe `checker` navodi `get_current_user` kao svoju ovisnost, pa se provjera identiteta izvede prije provjere uloge. Rutama je time dovoljno navesti samo `require_role`.

Uporaba u ruti

```
# app/routers/clubs.py
@router.post("/", response_model=ClubResponse, status_code=201)
async def create_club(
    body: ClubCreate,
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_role("admin")),
):
    ...

@router.get("/", response_model=list[ClubResponse])
async def list_clubs(
    db: AsyncSession = Depends(get_db),
    user: User = Depends(require_role("admin", "club")),
):
    ...
```

Iz potpisa rute vidi se tko je smije pozvati: stvaranje kluba samo organizator, popis klubova i organizator i klub.

Vlasništvo nad zapisom

Zašto ne stoji u ruti, provjera u servisu, sužavanje popisa, zaobilazanje kroz oznaku u putanji.

Zašto vlasništvo ne stoji u ruti

Provjera uloge gleda samo korisnika, pa se može izvesti prije nego zahtjev dođe do podataka. Provjera vlasništva to ne može, iz dvaju razloga:

- **Zapis mora biti dohvaćen.** Da bi se usporedilo pripada li klub korisniku, klub treba pročitati iz baze. Ruta to ne radi.
- **Pravilo nije isto za svaku radnju.** Klub smije vidjeti svoje podatke, ali ih ne smije mijenjati; organizator smije oboje. To nije svojstvo putanje nego operacije.

Vlasništvo je time poslovno pravilo, a ne svojstvo HTTP zahtjeva, pa stoji u sloju services, uz ostala pravila.

Provjera u servisu

```
# app/services/club_service.py
async def _get_or_404(db, club_id: int) -> Club:
    club = await club_repo.get_by_id(db, club_id)
    if not club:
        raise AppError("not_found", "Klub nije pronaden", 404)
    return club

async def get_club(db, club_id: int, current_user: User) -> Club:
    club = await _get_or_404(db, club_id)
    if current_user.role == "club" and current_user.club_id != club_id:
        raise AppError("forbidden", "...", 403)
    return club
```

Redoslijed nije slučajan: prvo se utvrdi postoji li zapis, pa tek onda smije li ga pošiljatelj vidjeti. Uvjet se odnosi samo na ulogu club, jer organizator smije vidjeti svaki klub, pa se za njega ne provjerava ništa.

Sužavanje popisa

Zahtjev za popisom ne imenuje nijedan zapis, pa nema zapisa za koji bi se moglo reći da je tuđi. Umjesto odbijanja, odgovor se sužava:

```
# app/services/club_service.py
async def list_clubs(db, current_user: User) -> list[Club]:
    if current_user.role == "admin":
        return await club_repo.get_all(db)
    club = await club_repo.get_by_id(db, current_user.club_id)
    return [club] if club else []
```

Ista ruta, isti statusni kod, različit sadržaj. Klub ne dobiva 403 nego popis od jednog zapisa.

To je i razlog zašto popis nikada ne smije dohvatiti sve pa filtrirati u odgovoru: podatci koji ne pripadaju korisniku ne izlaze iz sloja koji ih čita.

Izmjena oznake u adresi

Klub 12 otvori popis svojih natjecatelja. Preglednik pošalje:

```
GET /clubs/12/lifters
Authorization: Bearer <token kluba 12>
```

Isti zahtjev može se poslati i izvan sučelja, s jednom promijenjenom znamenkom:

```
GET /clubs/13/lifters
Authorization: Bearer <token kluba 12>
```

Token je valjan, nije istekao i uloga je club, kakvu ruta i traži. Provjera uloge dakle prolazi. Bez usporedbe kluba iz tokena s klubom iz putanje, odgovor sadrži natjecatelje kluba 13.

Takav se propust naziva nesigurno izravno pozivanje na zapis (engl. *insecure direct object reference*, IDOR).

Što ga ne sprječava

- **Sučelje koje tu adresu ne nudi.** Zahtjev ne mora doći iz sučelja. Alat naredbenog retka ili razvojna konzola preglednika šalju ga jednako dobro, s istim tokenom.
- **Oznaka koju je teško pogoditi.** Dovoljno ju je jednom vidjeti, a redni brojevi se ionako pogađaju redom. Nasumična oznaka usporava pretragu, ali ne mijenja ishod kad je oznaka poznata.
- **Provjera samo u putanji.** Oznaka kluba može stići i u tijelu zahtjeva, primjerice pri stvaranju natjecatelja. Provjeri li se samo putanja, tijelo ostaje nepokriveno.

Sprječava ga jedino usporedba pošiljatelja sa zapisom na koji se zahtjev odnosi, pri svakom zahtjevu i nad svakom oznakom koju je zahtjev donio.

Odgovori i njihove posljedice

Razlika između 401 i 403, izbor između 403 i 404, provjere koje se ne oslanjaju jedna na drugu.

Kada 401, a kada 403

Kod	Stanje	Što klijentu preostaje
401	pošiljatelj nije utvrđen	prijaviti se ili obnoviti token
403	pošiljatelj je utvrđen	ništa, zahtjev neće proći

Razlika nije formalna nego mijenja ponašanje klijenta. Na 401 klijent posegne za refresh tokenom i ponovi zahtjev. Vрати li poslužitelj 401 ondje gdje je zapravo riječ o nedostatku prava, klijent obnovi token, ponovi zahtjev, dobije isto, i korisnik ostane bez objašnjenja.

U kodu je razlika vidljiva po tome tko podiže pogrešku: `get_current_user` vraća 401, a `require_role` i provjera vlasništva 403.

Odati postojanje zapisa ili ne

Klub traži podatke kluba koji postoji, ali nije njegov. Bramo odgovoriti s:

- 403: zapis postoji, ali nemaš pravo na njega
- 404: kao da zapisa nema

Prvi odgovor odaje da zapis postoji. Nizom zahtjeva s različitim oznakama dobiva se popis onoga što u sustavu postoji, bez ijednog dohvaćenog podatka.

Izbor, ne pravilo

Za podatke kojima samo postojanje nije tajna, 403 je jasniji jer korisniku kaže što se dogodilo. Ondje gdje je i postojanje zapisa osjetljivo, 404 odaje manje. U ovom projektu klubovi su javno poznati, pa se vraća 403.

Zaštita u više slojeva

Mjesto	Što zaustavlja
sučelje	radnju koju korisnik ne treba vidjeti
routers	zahtjev bez tokena i pogrešnu ulogu
services	zahtjev nad tuđim zapisom
baza	zapis koji krši ograničenje

Ne smijemo se oslanjati na zaštitu preko sučelja: ono što ne prikaže i dalje se može poslati izravno na API.

Provjera uloge ne zaustavlja dohvat tuđeg zapisa, a ograničenje u bazi ne zna ništa o tome tko je poslao zahtjev.

Zahtjevi koji ne bi smjeli proći

Rutu obično isprobavamo kao korisnik koji na nju ima pravo, i tada sve radi. Izostane li provjera prava, ništa se ne mijenja: zahtjev i dalje prolazi, a odgovor izgleda jednako. Propust se pokaže tek kad isti zahtjev pošaljemo kao korisnik koji na njega nema pravo.

Zato za svaku rutu moramo testirati:

- bez tokena ili s neispravnim tokenom, uz očekivani 401
- s tokenom korisnika pogrešne uloge, uz očekivani 403
- s tokenom korisnika ispravne uloge, ali nad tuđim zapisom, uz očekivani 403

Treći je slučaj najlakše previdjeti, jer je zahtjev u svemu ispravan osim u jednom broju. Taj je propust ujedno i najčešći.

Sažetak

- Autorizacija odgovara na dva pitanja: smije li korisnik ovakvu radnju i smije li je nad ovim zapisom.
- Uloga se provjerava prije dohvata podataka, u ovisnosti uz rutu; vlasništvo tek nakon što je zapis pročitao, u sloju poslovnih pravila.
- Uloga koja ulazi u odluku dolazi iz baze, a ne iz tokena, pa promjena prava vrijedi odmah.
- Za popise nema što odbiti, pa se odgovor sužava: ista ruta, isti kod, različit sadržaj.
- 401 znači da se treba prijaviti, 403 da prijava neće pomoći; izbor između 403 i 404 odlučuje o da li se postojanje zapisa.